

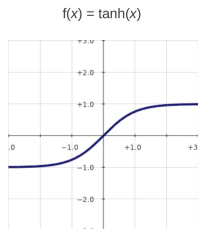
Training Tricks for CNNs + PyTorch

IALAB UC

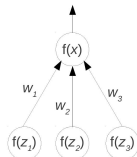
Computer Science Department, PUC

- Training Tricks
 - ReLUs
 - Dropout
 - Batch Normalization
- PyTorch - in depth!

Training Tricks: Rectified Linear Units (ReLUs)

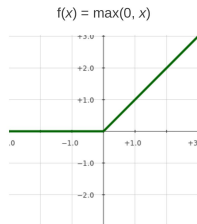


Very bad (slow to train)



$$x = w_1 f(z_1) + w_2 f(z_2) + w_3 f(z_3)$$

x is called the total input to the neuron, and $f(x)$ is its output

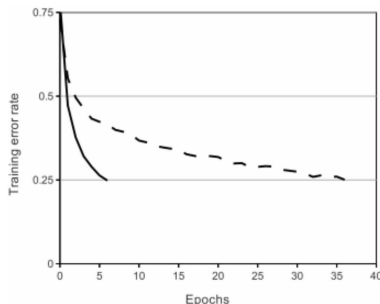


Very good (quick to train)

- Usually, NNs use sigmoid activation functions, such as $\tanh(x)$ or $(1 + \exp^{-x})^{-1}$. Here, however, they use Rectified Linear Units (ReLU).
- **Empirical observation:** Deep convolutional neural networks with ReLUs train several times faster than their equivalents with sigmoid units.

Training Tricks: Rectified Linear Units (ReLUs)

- Example: A four-layer convolutional neural network with ReLUs (solid line) reaches a 25% training error rate on CIFAR-10 six times faster than an equivalent network with $\tanh(x)$ neurons (dashed line).

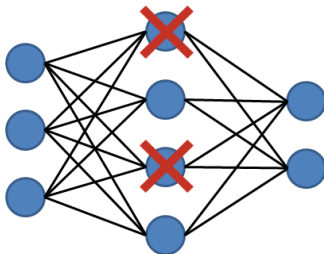


- In general, combining different models can be very useful (Mixture of experts, majority voting, boosting, etc.).
- Training many different models, however, is very time consuming.
- Here, they introduce Dropout.

¹Dropout: A Simple Way to Prevent Neural Networks from Overfitting
<http://jmlr.org/papers/v15/srivastava14a.html>

Training Tricks: Dropout

- At each iteration (epoch) set the output of each hidden neuron to zero with a certain probability.
- The neurons which are “dropped out” in this way do not contribute to the forward pass and do not participate in *backpropagation*.
- For every input, the neural network samples a different architecture, but all these architectures share weights.
- This technique reduces complex co-adaptations of neurons, since each neuron cannot rely on the presence of other neurons.



- Each neuron is forced to learn more robust features that are useful in conjunction with many different random subsets of other neurons.
- Without dropout, the network exhibits substantial *overfitting*.
- **Downside:** Dropout increases the number of iterations required to converge.

If you want to know of a few extensions to Dropout (which we won't cover here):

- **DropBlock**²: Dropping whole sets of neurons.
- **DropConnect**³: Dropping weights and biases randomly.

²DropBlock: A regularization method for convolutional networks <https://arxiv.org/abs/1810.12890>

³Regularization of Neural Networks using DropConnect
<http://yann.lecun.com/exdb/publis/pdf/wan-icml-13.pdf>

- As a ML problem, it is desirable that the distribution of training and test data matches.
- However, the distribution of the input to each layer changes during training (why?).
- This slows down the training by requiring lower learning rates and careful parameter initialization.
- We would like to keep the distribution of $\bar{x}$ fixed over time.
- Then, θ_2 does not have to readjust to compensate for the change in the distribution of $\bar{x}$.

⁴<https://arxiv.org/abs/1502.03167>

- BN takes a step towards reducing the internal covariate shift problem between layers.
- Specifically, BN introduces a normalization step that fixes the means and variances of layer inputs.

$$\hat{x}^{(k)} = \frac{\bar{x}^{(k)} - E[\bar{x}^{(k)}]}{\sqrt{\text{Var}[\bar{x}^{(k)}]}} \quad (1)$$

- Expectation and variance are computed over the mini-batch for each dimension k . So each dimension is normalized independently.
- Therefore, BN normalizes each scalar feature independently trying to make them unit gaussian, i.e., **each dimension has zero mean and unit variance**.

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_1 \dots x_m\}$;

Parameters to be learned: γ, β

Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$

$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$

A small constant ϵ is added to the denominator to avoid an eventual division by zero.

Batch Normalization (BN)

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_1 \dots x_m\}$;

Parameters to be learned: γ, β

Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$

$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$

- Simply normalizing each input of a layer might change the representational space of the layer.
- Ex. normalizing the inputs of a sigmoid would constrain them to lay in the linear part of the sigmoid.
- We need to add a mechanism to compensate for this effect.

- BN introduces a transformation that, if needed, allows the network to cancel the operation of the BN operator.
- In other words, the transformation provides the network with the flexibility to convert the BN operator in the identity function.
- Specifically, BN introduces for each activation $x^{(k)}$, parameters γ^k and β^k that scale and shift the normalized value.
- Using this transformation, if needed, the network can recover the original activations by setting $\gamma^k = \text{Var}[\bar{x}^{(k)}]$ and $\beta^k = E[\bar{x}^{(k)}]$.

Recall that:
$$\hat{x}^{(k)} = \frac{\bar{x}^{(k)} - E[\bar{x}^{(k)}]}{\sqrt{\text{Var}[\bar{x}^{(k)}]}}$$

- Parameters γ^k and β^k are learned during training.

Batch Normalization (BN)

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1\dots m}\}$;

Parameters to be learned: γ, β

Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$

$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$

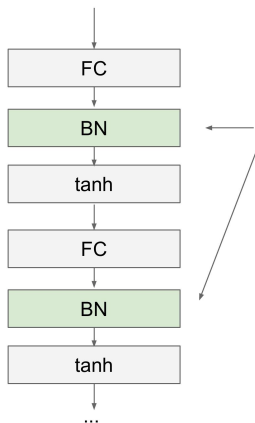
$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$

- What happens at test time?
- Do we have a mini batch?
- How mini-batch normalization operates at test time?

Mini-batches are normalized using the population, rather than mini-batch statistics.

Batch Normalization (BN)



Usually inserted after Fully Connected or Convolutional layers, and before nonlinearity.

$$\hat{x}^{(k)} = \frac{x^{(k)} - \mathbb{E}[x^{(k)}]}{\sqrt{\text{Var}[x^{(k)}]}}$$

Batch Normalization (BN)

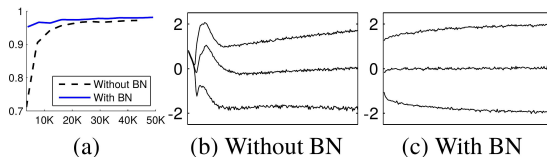


Figure 1: (a) *The test accuracy of the MNIST network trained with and without Batch Normalization, vs. the number of training steps. Batch Normalization helps the network train faster and achieve higher accuracy.* (b, c) *The evolution of input distributions to a typical sigmoid, over the course of training, shown as {15, 50, 85}th percentiles. Batch Normalization makes the distribution more stable and reduces the internal covariate shift.*

Model	Steps to 72.2%	Max accuracy
Inception	$31.0 \cdot 10^6$	72.2%
<i>BN-Baseline</i>	$13.3 \cdot 10^6$	72.7%
<i>BN-x5</i>	$2.1 \cdot 10^6$	73.0%
<i>BN-x30</i>	$2.7 \cdot 10^6$	74.8%
<i>BN-x5-Sigmoid</i>		69.8%

Figure 3: *For Inception and the batch-normalized variants, the number of training steps required to reach the maximum accuracy of Inception (72.2%), and the maximum accuracy achieved by the network.*

How Does Batch Normalization Help Optimization? (No, It Is Not About Internal Covariate Shift)

Shibani Santurkar*
MIT
shibani@mit.edu

Dimitris Tsipras*
MIT
tsipras@mit.edu

Andrew Ilyas
MIT
ailyas@mit.edu

Aleksander Mądry
MIT
madry@mit.edu